



Stack Overflow Sold to Prosus
Stack Overflow, an online community for software developers Sold to Prosus for \$1.8 Billion.
<http://digit.in/jul21-53>



Ohio sues Google
Ohio Attorney has filed a lawsuit to declare Google a public utility.
<http://digit.in/jul21-54>

How to rank searches

Information Retrieval systems scan through a large pool of data to pick out the most important results. Here how to make your own search engine.

Sachin Varlekar | feedback@digit.in

N

ow here the word “relevance” is the most subjective notion. Assume an ideal scenario given a

particular user and a large pool of data often called as “Corpus” what you want to figure out is something that is relevant and is very important to that person. For example if a user is searching for “Sachin Tendulkar” on a search engine. The Master Blaster of cricket Sachin Tendulkar will definitely appear on top of the search result. But there might be some other Sachin Tendulkar but he is a good surgeon. But most search engine would still rank cricketer Sachin Tendulkar on the top, that is because the search engines assume that notion of relevance is a notion that captures relevance for lot of people.

To summarise the information retrieval architecture in brief, there are whole bunch of *Documents* from which you have to retrieve the information. And this document has to be indexed. *Indexers* review documents and compile indexes to make information searches easier. They spend much of their time reviewing these documents or files so they

can use them for indexing purposes. Indexing is important because when a user type a query to gain some kind of information of the collection of document, one has to go over these documents very quickly and come over the answer within few seconds. For example say if you are searching for flight from Delhi to Goa on Google, you don’t want the answer to come up like after a minute or an hour. Answer has to be retrieved in a sub-second label. Therefore indexing is a very important aspect in Information Retrieval. Indexing means a compressed version of a document provided. So searching on index becomes much easier because you list out all the terms separately and duplicates from the contents of the documents are being removed. Now when the *User Query* comes in, such as for instance “Flight from Delhi to Goa”, this index is queried and a few important documents are obtained which matches this particular query. Then we come across *Ranking Model* which will try to rank among the top few obtained document result to determine which document is the most relevant from the list of all returned result from the query. When we search for Flights on Google the result obtained should be obviously from the relevant website say make-mytrip.com or Air India. There are chances it might also search for some other kind of news article related to the query “Delhi to Goa” but what we are searching over here is for flights and not news article. So relevance assessment is also another important aspect of Information retrieval.

Information Retrieval is not only about web search but also about other use cases such as Desktop Search. For example when we try to search for some application or a file or folder on Windows operating

system start menu search bar, among all the file names one has to figure out those names that match what you are searching for. It can also be used for library search. In 1960’s and 1970’s researchers made use of Information Retrieval Techniques for patent search. Given a large collection of patents how do you figure out which of those patents, contain the query words that you are looking for? Therefore, you can say that Information Retrieval concept was discovered way back in 1960’s.

INVERTED INDEX

An inverted index is an index data structure storing a mapping from content, such as words or numbers, to its locations in a document or a set of documents. Inverted index allows fast full text searches, at a cost of increased processing when a document is added to the database. It is also easy to develop. It is the most popular data structure used in document retrieval systems, used on a large scale in search in most of the search engines.

Example

Consider these documents:

- Doc 1 - breakthrough drug for schizophrenia
- Doc 2 - new schizophrenia drug
- Doc 3 - new approach for treatment of schizophrenia
- Doc 4 - new hope for schizophrenia patients

If we want to build an inverted index for these documents it will be as follows:

So if you observe this illustration, this is how inverted index stores data. All the sentences from the list of given documents are being broken down one by one in the form of single words and stored. This procedure is referred to as tokenization. The terms should not be redundant. Terms are